

Neurons, Activations, and Feedforward Nets

DATA 443/643 · Week 1, note 1.2

After MIT 6.S191 L1 and CS231N L4: one unit, then why depth needs a bend.

Lecture plan

- 1 One neuron: z , then a
- 2 Four formulas you will write
- 3 Why two linear layers collapse
- 4 XOR, Goodfellow's table, 3 vs 33

First time: what / why / formula

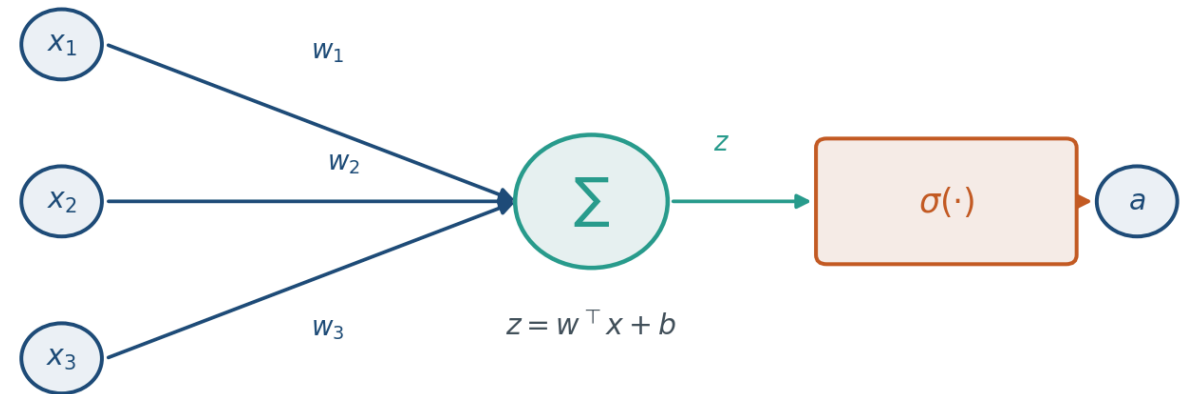
- What / why. A weighted sum plus a bias, then a nonlinearity. Stack them and you have an MLP. Linear maps compose to one linear map. The bend is what makes depth useful.
- Architecture. Input $x \rightarrow z = w^\top x + b \rightarrow a = \sigma(z)$. An MLP stacks that layerwise.
- Formula. $z = w^\top x + b$, $a = \sigma(z)$, $\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}$.
- Tradeoffs. + ReLU is cheap and sparse. – Sigmoid saturates (bad gradients). No nonlinearity $\Rightarrow$ no extra function class.

How. Pick σ (ReLU in hidden layers, softmax when classes compete). Train w, b next note.

One neuron

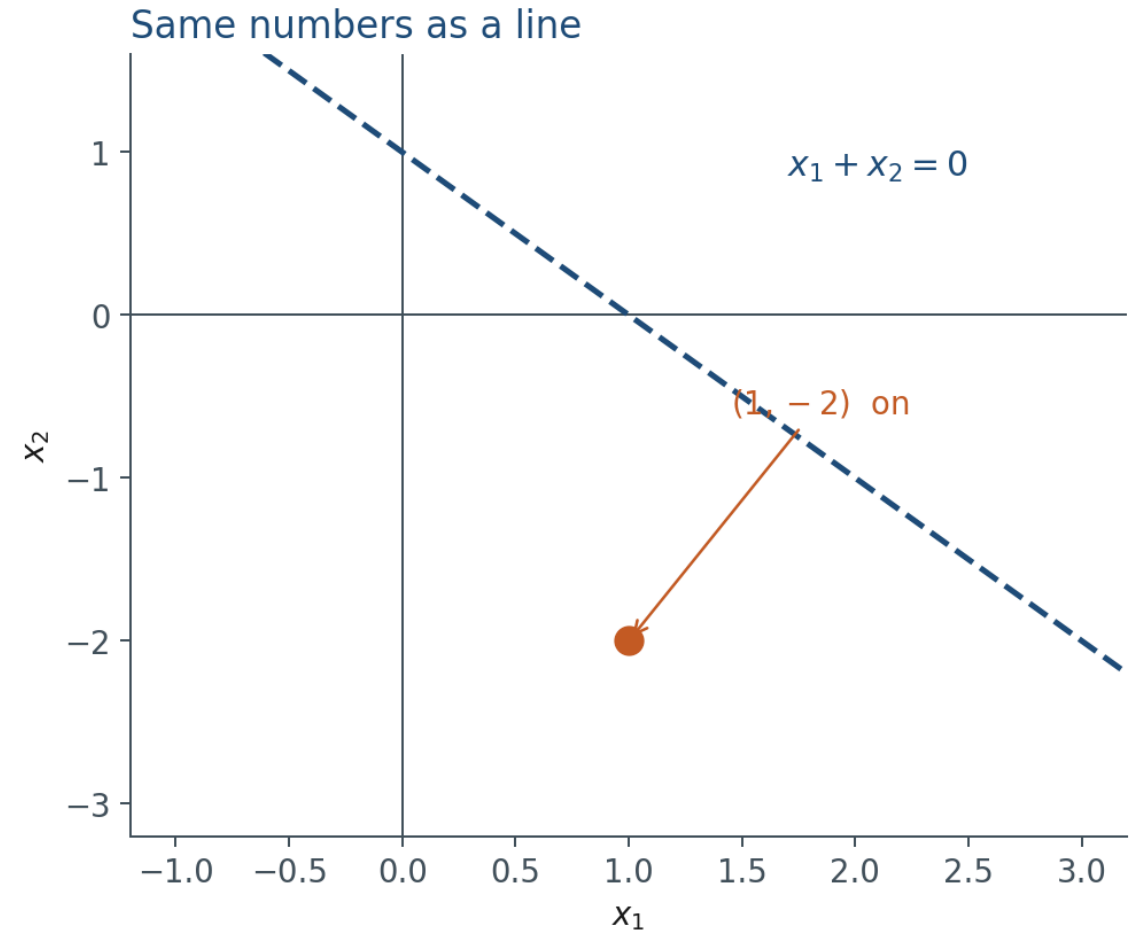
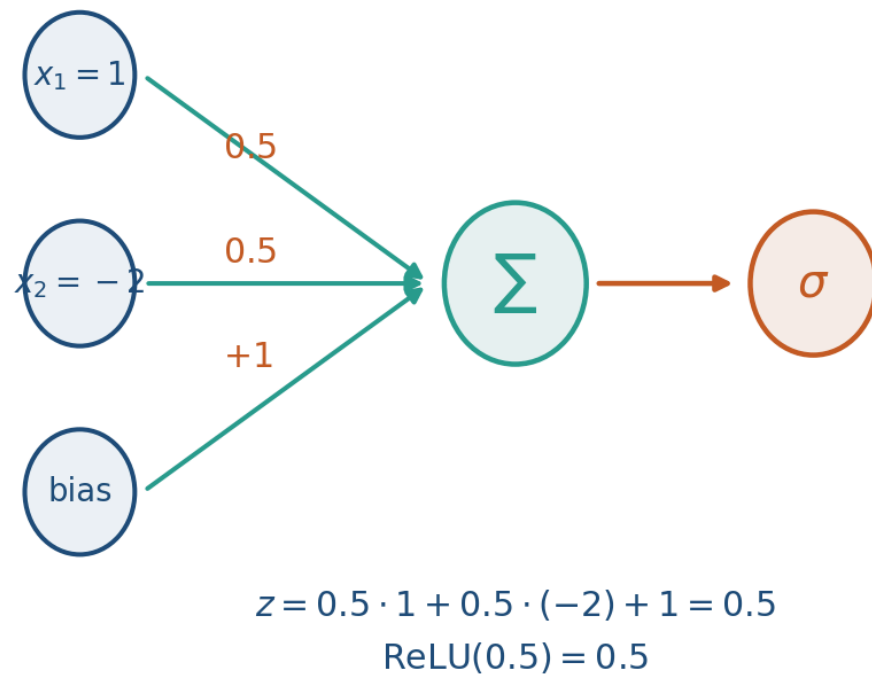
- Pre-activation $z = w^T x + b$.
- Activation $a = \sigma(z)$.
- If you skip z , you cannot take derivatives next note.

One neuron: weighted sum, then a nonlinearity



The perceptron: plug in the numbers

Plug in the numbers



MIT 6.S191 style: same example as a diagram and as a line. Ours uses ReLU, not sigmoid.

Four formulas. No Jacobian today.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{ReLU}(z) = \max(0, z)$$

$\text{ReLU}'(z) = 1_{z > 0}$. At 0, PyTorch uses 0.

$\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}$.

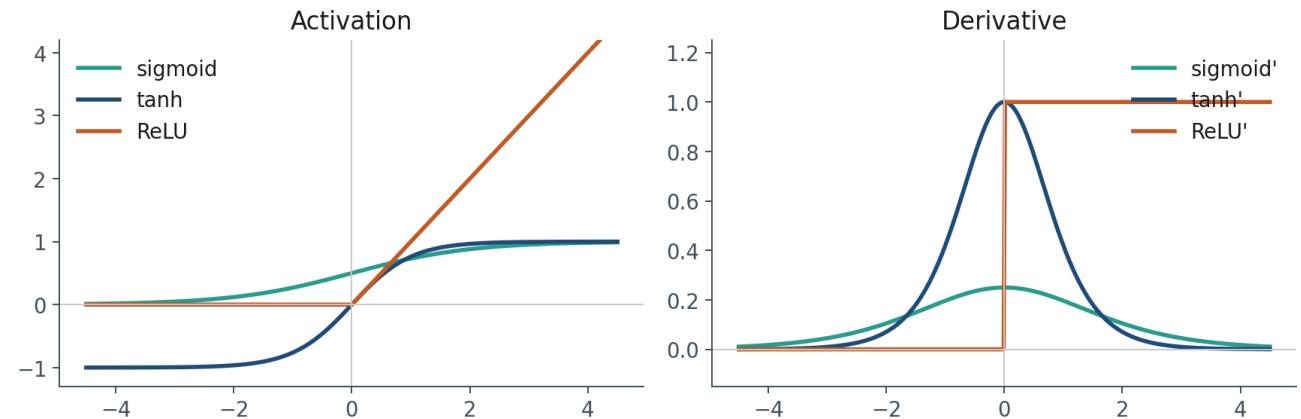
ReLU in hidden layers. Softmax when classes compete.

The nonlinearity is the point

- Two affine maps compose to one affine map.
- Without a bend, depth adds no power.
- CS231N: you get a linear classifier again.

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

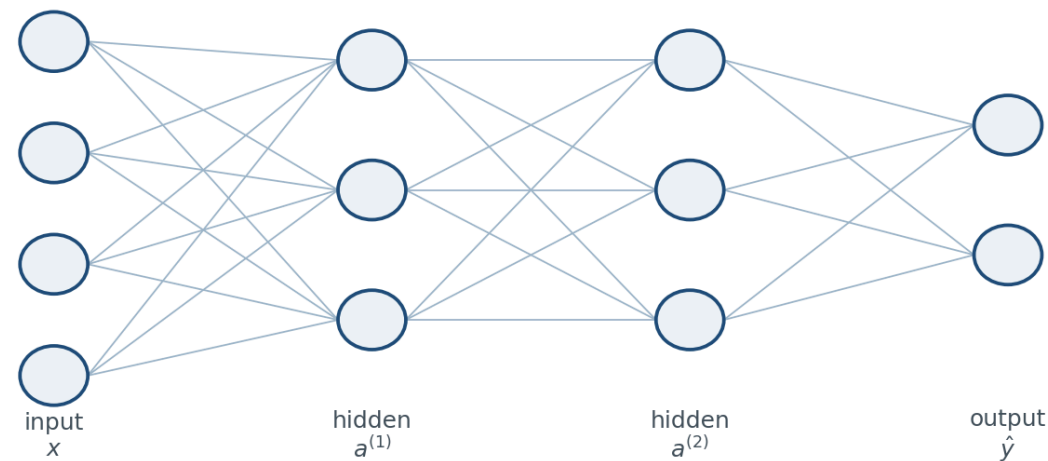
Without a nonlinearity, stacked layers collapse to one linear map



A feedforward net

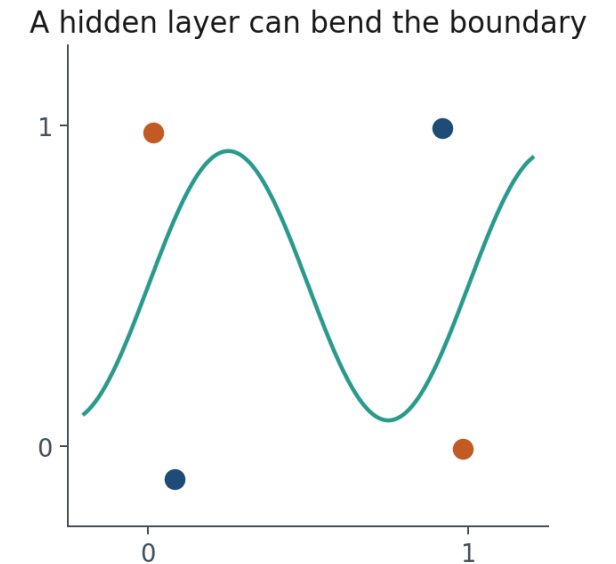
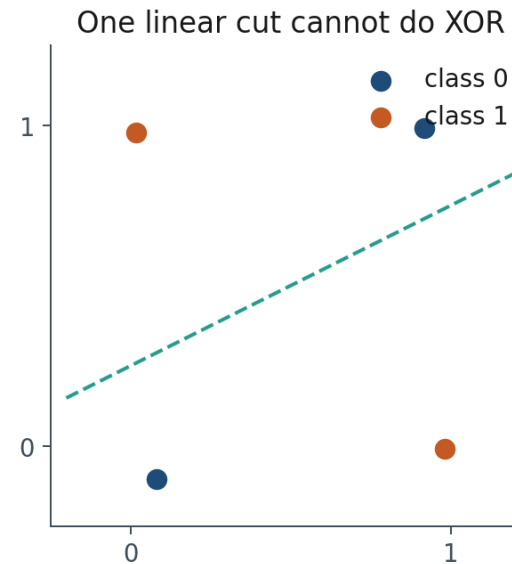
- Layer ℓ : $a^{(\ell)} = \sigma(W^{(\ell)}a^{(\ell-1)} + b^{(\ell)})$.
- $a^{(0)} = x$. Every arrow is one weight.
- A transformer block still contains this MLP.

A feedforward net: each layer is $a^\ell = \sigma(W^\ell a^{\ell-1} + b^\ell)$



XOR: why a hidden layer exists

- One linear unit cannot separate XOR.
- A hidden layer folds the square.
- Language models stack many such maps.



Goodfellow §6.1: an explicit XOR net

XOR with two ReLUs: OR, AND, then subtract (Goodfellow 6.1)

(x_1, x_2)	$a_1 = \text{ReLU}(x_1 + x_2)$	$a_2 = \text{ReLU}(x_1 + x_2 - 1)$	$\hat{y} = a_1 - 2a_2$
(0, 0)	0	0	0
(0, 1)	1	0	1
(1, 0)	1	0	1
(1, 1)	2	1	0

Lab 1 MLP is 33 parameters, not this hand-chosen net. Linear(2,1) is 3 parameters and cannot do this table.